

PROJECT 1

Adaptive 3D Gaussian Splatting for Real-Time Online 3D Reconstruction

Advanced Machine Learning Core + AI Systems Optimization Layer

Project thesis

Trọng tâm của dự án là Advanced Machine Learning: hiểu, cài đặt và cải tiến 3D Gaussian Splatting như một bài toán học biểu diễn 3D, differentiable rendering và online optimization. Phần C++/CUDA/GPU runtime được xem là lớp phát triển thêm để giảm latency, memory và computation, chứ không thay thế contribution ML.

Baseline paper: RTG-SLAM (SIGGRAPH 2024)

Source file provided: 2404.19706v1.pdf

Version: Project design / study roadmap

1. Executive Summary

Mục tiêu. Xây dựng một hệ thống tái tạo và render môi trường 3D bằng 3D Gaussian Splatting (3DGS), trong đó phần cốt lõi tập trung vào Advanced Machine Learning: scene representation, covariance parameterization, differentiable projection/splatting, RGB/depth rendering, loss design, online Gaussian optimization và adaptive densification. Sau đó phát triển một lớp AI Systems để thực thi các phép tính này hiệu quả trên GPU và giữ thời gian thực.

Tầng	Mục tiêu	Thành phần chính	Vai trò
ML / Research core	Hiểu và cải tiến representation + optimization	3DGS, SH, differentiable rendering, RGB-D losses, uncertainty, Gaussian importance	Bắt buộc
Online reconstruction	Học và cập nhật scene theo stream	Gaussian addition, pruning, stable/unstable or continuous confidence, keyframes, tracking	Bắt buộc
AI Systems	Giảm latency / memory / compute	C++/CUDA, tiling, radix sort, fusion, memory layout, scheduling	Phát triển thêm
Runtime / Demo	Hệ thống tương tác	Vulkan/OpenGL viewer, camera control, profiling, benchmark	Đầu ra kỹ thuật

Một câu để ghi vào portfolio

Designed and implemented an online 3D Gaussian Splatting reconstruction pipeline, studying differentiable RGB-D optimization and adaptive Gaussian selection, then accelerated the reconstruction/rendering runtime with custom C++/CUDA kernels under an explicit latency and GPU-memory budget.

2. Research Question and Positioning

2.1. Câu hỏi trung tâm

Câu hỏi ML: Làm thế nào để dùng một tập Gaussian 3D compact nhưng vẫn đủ biểu diễn geometry + appearance, và cập nhật các Gaussian đó online từ RGB-D stream mà không phải tối ưu toàn bộ scene ở mỗi frame?

Câu hỏi systems mở rộng: Khi GPU có một ngân sách thời gian cố định, Gaussian nào cần được render, optimize hoặc densify để tối đa hóa chất lượng dưới constraint về latency và memory?

Định vị dự án

Đây không phải project "load .ply và viết renderer". Renderer là một phần. Cốt lõi là nghiên cứu representation + optimization policy cho 3DGS online; CUDA/Vulkan được dùng để đưa thuật toán đó thành một hệ thống real-time có thể đo lường.

3. Background: From NeRF to 3DGS

3.1. NeRF

NeRF biểu diễn scene bằng một hàm neural implicit: $F_\theta(x, d) \rightarrow (\sigma, c)$, trong đó x là vị trí 3D, d là viewing direction, σ là density và c là color. Novel-view synthesis được thực hiện bằng volumetric rendering dọc theo ray. Điểm yếu chính đối với online real-time reconstruction là cost của volume rendering và optimization.

3.2. 3DGS. Thay vì một MLP implicit, scene được biểu diễn bằng nhiều Gaussian primitives. Mỗi Gaussian có vị trí, covariance, opacity và các hệ số Spherical Harmonics (SH). Covariance thường được parameterize bằng scale + rotation để đảm bảo tính hợp lệ.

$$G_i = (\mu_i, \Sigma_i, \alpha_i, SH_i)$$

$$\Sigma_i = R_i S_i S_i^T R_i^T$$

Ý nghĩa: Gaussian có thể trở thành một ellipsoid anisotropic, do đó một primitive có thể phủ một local surface region hiệu quả hơn một point đơn thuần.

4. Advanced Machine Learning Core

4.1. Gaussian representation

Tham số	Vai trò	ML implication	Có thể cải tiến
$\mu \in \mathbb{R}^3$	Vị trí Gaussian	Geometry / scene structure	Uncertainty-aware update
$\Sigma \in \mathbb{R}^{3 \times 3}$	Shape / orientation / extent	Surface approximation	Anisotropic regularization
α	Opacity / visibility contribution	Compositing	Adaptive opacity policy
SH coefficients	View-dependent appearance	Photometric representation	Adaptive SH degree / view complexity
normal	Surface orientation (khi dùng RGB-D/surfel)	Geometry supervision	Normal consistency
confidence / state	Mức tin cậy của Gaussian	Optimization selection	Continuous importance score

4.2. Projection từ 3D sang screen space

Camera transform: $x_c = W x$. Perspective projection: $p = \pi(x_c)$. Vì projection là phi tuyến, covariance screen-space được xấp xỉ bằng Jacobian:

$$\Sigma_{2D} = J W \Sigma W^T J^T$$

Đây là bước quan trọng để chuyển Gaussian 3D thành elliptical footprint trên màn hình. Nó vừa là toán học của model, vừa là dữ liệu đầu vào cho rasterization.

4.3. Differentiable splatting

$$f_i(u) = \alpha_i \cdot \exp(-1/2 \cdot (u - \mu_i)^T \Sigma_{2D_i}^{-1} (u - \mu_i))$$

Color được alpha-composite từ front-to-back:

$$C(u) = \sum_i c_i f_i(u) \prod_{j < i} (1 - f_j(u))$$

Light transmission: Trong RTG-SLAM, các quantity này được dùng không chỉ để render mà còn để tạo mask cho Gaussian addition và xác định vùng cần tối ưu. *Ngcite-placeholder*

Trong RTG-SLAM, color, depth, light transmission và Gaussian index map được dùng để quyết định nơi cần thêm Gaussian và nơi cần tối ưu; đây là cầu nối trực tiếp giữa differentiable rendering và online learning.

4.4. RGB-D supervision

Với RGB-D input, project có hai nguồn supervision chính: photometric và geometric.

$$L_{color} = |C_{gt} - C_{render}|$$
$$L_{depth} = |D_{gt} - D_{render}|$$
$$L = w_c L_{color} + w_d L_{depth} + regularization$$

Điểm mạnh của RGB-D là geometry có thể được giám sát trực tiếp thay vì chỉ suy ra từ photometric consistency.

4.5. Depth rendering theo RTG-SLAM

Insight quan trọng của paper: Không dùng alpha blending của color để lấy depth. Với mỗi pixel, lấy Gaussian opaque đầu tiên vượt ngưỡng opacity, coi nó như một ellipsoid disc trên dominant plane và tính ray-plane intersection. Cách này giúp một Gaussian opaque có thể fit một local surface region mà không cần nhiều Gaussian chồng lấn.

$$\theta = ((p_G - t_{cam}) \cdot n_G) / (ray_{dir} \cdot n_G)$$
$$p_{hit} = t_{cam} + \theta * ray_{dir}$$
$$D(u) = depth(p_{hit})$$

5. Paper Foundation: RTG-SLAM

Paper: Zhexi Peng et al., “RTG-SLAM: Real-time 3D Reconstruction at Scale using Gaussian Splatting”, SIGGRAPH 2024, arXiv:2404.19706v1.

Paper đặt mục tiêu đưa Gaussian Splatting vào online RGB-D reconstruction ở quy mô lớn. Hai ý tưởng nổi bật là compact Gaussian representation và highly efficient on-the-fly Gaussian optimization. Paper báo cáo 17.9 FPS và 8.8 GB memory trên Azure Home trong setup của họ, cùng custom CUDA kernels cho rasterization và backpropagation.

Cơ chế RTG-SLAM	Ý tưởng	Tại sao hữu ích
Opaque / transparent Gaussian	$\alpha \approx 0.99$ cho surface + dominant color; $\alpha \approx 0.1$ cho residual color	Giảm số Gaussian cần cho geometry, giữ khả năng sửa appearance
Custom depth rendering	Ray-plane intersection với frontmost opaque Gaussian	Một Gaussian có thể fit local plane tốt hơn depth alpha blending
Error masks	Color error + depth error + transmission	Xác định nơi scene đang thiếu geometry hoặc appearance
Selective Gaussian addition	Chỉ sample một phần pixel trên error masks	Giảm memory/computation khi mapping online
Stable / unstable state	Chỉ optimize Gaussian chưa ổn định	Giảm số parameter cần backprop
Selective pixel rendering	Chỉ render pixel bị unstable Gaussians ảnh hưởng	Giảm computation trong optimization
ICP + graph optimization	Tracking bằng frame-to-model ICP và backend landmark graph	Giữ camera pose ổn định khi scan scene lớn

6. Full Project Architecture

OFFLINE / INITIALIZATION

Images or RGB-D sequence

- > camera calibration / poses
- > initial point cloud / initial Gaussians

ONLINE

RGB-D frame k

- > camera tracking
- > current Gaussian render
- > color/depth/transmission/index errors
- > Gaussian addition / pruning
- > Gaussian importance estimation
- > selective optimization
- > updated Gaussian map
- > next frame

RUNTIME LAYER

Gaussian map

- > GPU-friendly storage
- > CUDA projection / binning / sorting / rasterization
- > Vulkan/OpenGL presentation

Phân tách quan trọng

Advanced ML là core: representation, losses, depth rendering, uncertainty/importance, online optimization và densification policy. AI Systems là acceleration layer: data layout, kernels, sorting, scheduling, memory management, GPU profiling và rendering runtime.

7. Proposed Research Novelty

7.1. Vì sao không nên chỉ reproduce RTG-SLAM

RTG-SLAM đã có compact Gaussians, RGB-D online optimization, stable/unstable selection và custom CUDA rasterization/backpropagation. Vì vậy, một project chỉ re-implement những thành phần này sẽ có giá trị học tập tốt nhưng novelty nghiên cứu thấp. Contribution mới nên nằm ở policy quyết định "Gaussian nào cần compute và compute bao nhiêu".

7.2. Continuous Gaussian Confidence / Importance

Thay vì binary stable/unstable, xây một score liên tục $q_i \in [0,1]$ hoặc importance I_i .

$$\begin{aligned}
 I_i = & w_g * E_{\text{depth}_i} \\
 & + w_p * E_{\text{color}_i} \\
 & + w_n * E_{\text{normal}_i} \\
 & + w_v * \text{Visibility}_i
 \end{aligned}$$

+ $w_t * \text{TemporalChange}_i$
 + $w_s * \text{ScreenSpaceImportance}_i$

Score này quyết định mức độ cập nhật: Gaussian quan trọng cao được optimize thường xuyên; score trung bình được optimize định kỳ; score thấp được freeze nhưng vẫn có thể render.

7.3. Budget-aware Gaussian Scheduler

Đây là contribution AI Systems mở rộng, nhưng nó cũng tạo ra một bài toán ML/system co-design.

maximize $\sum_{i \in S} \text{Importance}_i$
 subject to $\sum_{i \in S} \text{Cost}_i \leq \text{GPU_Budget}$

GPU_Budget có thể là số milliseconds cho mapping/optimization trong mỗi frame, ví dụ 2-4 ms.

Scheduler phân bổ ngân sách cho: (1) Gaussian optimization, (2) densification, (3) rendering quality/LOD, (4) memory movement.

7.4. Adaptive Thresholds

RTG-SLAM sử dụng các threshold cố định cho transmission, depth error và color error trong thực nghiệm. Project có thể điều chỉnh threshold theo noise statistics, scene complexity và GPU budget.

$\text{delta_depth}(t) = k * \text{sigma_depth}(t)$
 $\text{delta_color}(t) = k * \text{sigma_color}(t)$
 $\text{threshold} = f(\text{scene_complexity}, \text{uncertainty}, \text{GPU_budget})$

7.5. Error-driven LOD

Không chỉ distance-based LOD. Một Gaussian ở mép vật thể hoặc vùng texture phức tạp có thể quan trọng hơn một Gaussian trên bức tường phẳng dù cùng khoảng cách đến camera. Dùng score này để preserve detail ở vùng có information content cao.

$\text{LOD_score}_i = \text{ScreenSpaceArea}_i * \text{PhotometricError}_i * \text{GeometricComplexity}_i$

7.6. Optional: Online memory tiering

Khi scene lớn hơn VRAM, có thể chia Gaussian thành hot/warm/cold tiers: GPU VRAM, CPU RAM và storage. Đây nên là phase sau, chỉ triển khai khi core ML đã ổn định.

8. Detailed ML Pipeline

8.1. Initialization

1. Từ camera calibration + camera poses, back-project RGB-D pixels thành 3D points.
2. Khởi tạo Gaussian position từ 3D points; normal từ local geometry.
3. Khởi tạo covariance theo local tangent plane: hai trục lớn trên surface và một trục nhỏ theo normal.
4. Khởi tạo color/SH từ RGB observation và opacity theo policy.

8.2. Forward rendering

5. Transform Gaussian center và covariance vào camera frame.
6. Project sang screen space bằng Jacobian.
7. Compute screen-space ellipse / bounding box.
8. Bin Gaussian vào tile.

9. Sort các Gaussian theo tile + depth.
10. Rasterize Gaussian và alpha-composite color.
11. Render depth bằng surface-aware intersection của opaque Gaussian.
12. Sinh transmission map, normal map và Gaussian index map.

8.3. Online update

13. Track camera pose bằng RGB-D ICP hoặc một frontend tương đương.
14. Render current map ở pose hiện tại.
15. Tính color error, depth error, normal consistency và transmission.
16. Sinh candidate masks cho Gaussian addition.
17. Densify / add Gaussian theo score thay vì uniform-only sampling.
18. Tính importance/confidence cho Gaussian.
19. Chọn subset để optimize dưới budget.
20. Cập nhật parameters bằng gradient descent.
21. Prune outlier / low-value Gaussian theo long-term statistics.

9. Learning Objective and Optimization

9.1. Baseline objective

$$L = w_c L_{\text{color}} + w_d L_{\text{depth}} + w_n L_{\text{normal}} + w_{\text{reg}} L_{\text{reg}}$$

Trong phiên bản baseline, có thể bắt đầu với L1 cho color/depth như paper. Sau đó mới mở rộng sang robust loss (Charbonnier/Huber), normal consistency và view-consistency.

9.2. Proposed adaptive regularization

$$L_{\text{total}} = L_{\text{rgbd}} + \lambda_{\text{geo}} L_{\text{geometry}} + \lambda_{\text{temp}} L_{\text{temporal}} + \lambda_{\text{comp}} L_{\text{compact}}$$

L_{compact} có thể phạt Gaussian dư thừa; L_{temporal} khuyến khích scene ổn định qua nhiều frame; L_{geometry} giữ normal/depth nhất quán. Không nên đưa tất cả loss vào ngay từ đầu; ablation phải chứng minh từng thành phần có tác dụng.

9.3. Optimization scheduling

Thay vì optimize toàn scene, dùng scheduler:

Tier	Điều kiện ví dụ	Hành động
Tier A	High importance + high error	Optimize every update
Tier B	Medium importance	Optimize every N frames
Tier C	Stable / low error	Freeze parameters
Tier D	Persistent low contribution / outlier	Prune or evict

10. AI Systems Optimization Layer

10.1. CUDA pipeline

Gaussian buffers

-> preprocess / project

-> frustum + contribution culling

Advanced Machine Learning core + AI Systems optimization layer

- > tile binning
- > prefix-sum / compaction
- > depth sorting
- > tile rasterization
- > color/depth/transmission
- > per-Gaussian statistics
- > scheduler feedback

10.2. Memory layout

Ưu tiên SoA hoặc hybrid layout thay vì AoS thuần. Tách buffers cho position, scale/rotation, opacity, SH và state metadata để kernel chỉ đọc dữ liệu cần thiết.

10.3. Sorting

Dùng CUB radix sort làm baseline. Sau khi pipeline đúng và profiling chỉ ra sorting là bottleneck, mới thử custom radix sort theo tile/depth workload.

10.4. Tile rasterization

Mapping tự nhiên: 1 CUDA block phụ trách một tile; threads phụ trách pixels. Shared memory có thể dùng để stage Gaussian batches khi reuse đủ cao.

10.5. Early termination

$\text{color} += T * \alpha * c$

$T *= (1 - \alpha)$

if $T < \epsilon$: break

Đây vừa là optimization GPU vừa ảnh hưởng trực tiếp chất lượng render, nên cần benchmark theo scene và threshold.

10.6. GPU profiling

Metric	Cần theo dõi
Latency	ms/frame, p50, p95
GPU utilization	SM utilization, occupancy
Memory	VRAM usage, bandwidth, L2 hit rate
Kernel	time per kernel, launch count
Control flow	warp divergence / branch efficiency
Scalability	frame time vs Gaussian count

11. Real-Time Viewer and Runtime

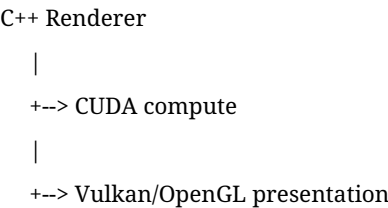
Viewer nên là phần demo cuối: camera tự do, WASD + mouse, render interactive. Có thể bắt đầu bằng OpenGL để đơn giản hóa, sau đó chuyển sang Vulkan nếu mục tiêu systems cần explicit GPU resource management và CUDA/Vulkan interop.

Application

Camera Controller

|

v



12. Experimental Design

12.1. Baselines

Baseline	Mục đích
Offline 3DGS renderer	Xác nhận quality của representation
RTG-SLAM style policy	Baseline online RGB-D
Binary stable/unstable	Baseline scheduling
Continuous importance	Proposed ML policy
Fixed thresholds	Baseline adaptation
Adaptive thresholds	Proposed policy
Uniform densification	Baseline Gaussian addition
Importance-driven densification	Proposed addition policy

12.2. Ablation studies bắt buộc

- 22. Không có confidence/importance -> binary stable/unstable.
- 23. Importance chỉ dùng color error.
- 24. Color + depth.
- 25. Color + depth + normal + temporal statistics.
- 26. Fixed GPU budget vs adaptive GPU budget.
- 27. Uniform densification vs error-driven densification.
- 28. No LOD vs error-driven LOD.

12.3. Metrics

Nhóm	Metric đề xuất
Novel view / appearance	PSNR, SSIM, LPIPS hoặc metric phù hợp dataset
Depth / geometry	Depth L1/AbsRel, Accuracy, Completion, Chamfer nếu có ground truth
Tracking	ATE / trajectory error
Systems	FPS, frame time, p50/p95 latency
Efficiency	Gaussian count, VRAM, peak memory, update rate
Robustness	camera speed, lighting/noise, depth sparsity, scene scale

13. Datasets and Demo Strategy

Theo paper, các benchmark đã dùng gồm Replica, TUM-RGBD, ScanNet++ và dữ liệu Azure tự scan. TUM-RGBD phù hợp tracking; Replica phù hợp iteration nhanh; ScanNet++ phù hợp large-scale geometry. Paper cũng mô tả self-captured scenes trong phạm vi khoảng 43-100 m².

Dataset / source	Vai trò trong project
Replica	Debug ML + ablation nhanh
TUM-RGBD	Camera tracking evaluation

ScanNet++	Geometry / scale evaluation
Self-captured RGB-D	Demo real-time end-to-end

14. Suggested Repository Structure

```

adaptive-3dgs/
├─ configs/
├─ datasets/
├─ research/
│   ├─ gaussian_repr.py
│   ├─ losses.py
│   ├─ importance.py
│   ├─ densification.py
│   └─ scheduler.py
├─ cuda/
│   ├─ preprocess.cu
│   ├─ binning.cu
│   ├─ radix_sort.cu
│   ├─ rasterize.cu
│   ├─ depth_render.cu
│   └─ statistics.cu
├─ cpp/
│   ├─ scene.cpp
│   ├─ renderer.cpp
│   └─ runtime.cpp
├─ viewer/
│   └─ vulkan_or_opengl/
├─ benchmarks/
├─ scripts/
└─ docs/

```

15. Development Roadmap

Phase	Nội dung	Exit criteria
0	Study 3DGS + RTG-SLAM, reproduce equations	Có notebook giải thích + unit tests cho core math
1	Static Gaussian renderer	Render được scene offline
2	RGB-D loss + depth rendering	Gradient update đúng, depth stable
3	Online Gaussian addition	Scene grows theo frame stream
4	Baseline RTG-SLAM policy	Có baseline metrics
5	Continuous importance / adaptive policy	Ablation chứng minh quality/compute tradeoff
6	CUDA acceleration	GPU pipeline đúng + profiler report
7	Budget-aware scheduler + LOD	Giữ target latency dưới workload tăng
8	Viewer + packaging	Demo end-to-end

9	Paper-style evaluation	Tables, plots, ablations, reproducibility
---	------------------------	---

16. Knowledge Checklist

Nhóm	Kiến thức cần nắm
3D Geometry	SE(3), camera intrinsics/extrinsics, projective geometry, ray casting
Probability / optimization	Gaussian distribution, covariance, gradient descent, regularization
Differentiable rendering	alpha compositing, Jacobian, screen-space covariance, ray-plane intersection
Advanced ML	loss design, online optimization, uncertainty, adaptive densification, temporal consistency
SLAM	ICP, pose estimation, keyframes, drift, graph optimization
CUDA	grid/block/warp, memory hierarchy, coalescing, shared memory, scans, sorting
Systems	profiling, scheduling, memory budget, latency, runtime design
Graphics	Vulkan/OpenGL, frame synchronization, GPU resource management

17. Risks and Scope Control

Risk	Cách kiểm soát
Quá rộng: SLAM + 3DGS + CUDA + Vulkan + novelty	Khóa ML baseline trước; systems là phase sau
Không tái tạo được paper ngay	Reproduce từng module: render -> depth -> addition -> optimization
CUDA optimization che mất ML	Mọi optimization phải gắn với một ML workload và có baseline
Novelty không đủ	Định nghĩa rõ proposed policy + ablation trước khi tối ưu low-level
Dataset quá lớn	Bắt đầu Replica/TUM, sau đó mới ScanNet++ và self-captured
Quality giảm khi scheduler tiết kiệm compute	Bắt buộc báo cáo Pareto curve: quality vs latency vs memory

18. Final Project Definition

Core research contribution

Một pipeline 3DGS online có khả năng học và cập nhật scene từ RGB-D stream; đóng góp chính tập trung vào compact/uncertainty-aware Gaussian representation, differentiable RGB-D optimization và adaptive Gaussian importance/densification policy.

AI Systems contribution

Một runtime C++/CUDA có budget-aware scheduling, efficient tile-based rasterization, selective optimization, memory-aware data layout và profiling; mục tiêu là đạt real-time với chất lượng được đo theo Pareto trade-off.

Thứ tự ưu tiên: 1) Advanced ML correctness -> 2) online optimization -> 3) proposed adaptive policy -> 4) CUDA acceleration -> 5) Vulkan/runtime polish. Nếu phải cắt scope, giữ 1-3 và bỏ 5 trước.

19. References / Source Basis

[1] Z. Peng, T. Shao, Y. Liu, J. Zhou, Y. Yang, J. Wang, K. Zhou. RTG-SLAM: Real-time 3D Reconstruction at Scale using Gaussian Splatting. SIGGRAPH Conference Papers 2024. arXiv:2404.19706v1. DOI: 10.1145/3641519.3657455.

[2] B. Kerbl, G. Kopanas, T. Leimkuehler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG 42(4), 2023.

[3] B. Mildenhall et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. ECCV 2020.

Ghi chú nguồn: phần phân tích chi tiết RTG-SLAM trong tài liệu này được tổng hợp trực tiếp từ file 2404.19706v1.pdf do người dùng cung cấp, đặc biệt các phần Method, Online Reconstruction, Evaluation và Conclusion. Các đề xuất Continuous Importance, Budget-aware Scheduling, Adaptive Thresholds và Error-driven LOD là thiết kế mở rộng cho project, không phải claim rằng chúng là contribution của RTG-SLAM.